

排序報告

排序演算法模擬實作報告

學生資訊

- 學號：XXXXXXX
 - 姓名：Jia-Yin
 - 模擬頁面：https://jia-yin.github.io/sort_report/
-

1. 氣泡排序法 (Bubble Sort)

原理

透過重複走訪陣列，比較相鄰的兩個元素，若順序錯誤則交換。每一輪走訪後，最大的元素會像「氣泡」一樣浮動到陣列末端。

複雜度分析

- 時間複雜度：最佳 $O(n)$ / 平均 $O(n^2)$ / 最差 $O(n^2)$
- 空間複雜度： $O(1)$

模擬實作內容

利用雙重迴圈控制，當 `data[j] > data[j+1]` 時執行交換。在模擬頁面中，會即時高亮顯示正在比較的相鄰色塊。

2. 選擇排序法 (Selection Sort)

原理

每一輪從「未排序部分」找出最小的元素，將其與未排序部分的第 1 個元素交換，直到所有元素都放入已排序部分。

複雜度分析

- 時間複雜度：最佳 $O(n^2)$ / 平均 $O(n^2)$ / 最差 $O(n^2)$
- 空間複雜度： $O(1)$

模擬實作內容

設定一個變數紀錄當前最小值的索引 (index)，掃描後方剩餘元素，最後僅執行一次交換動作。

3. 插入排序法 (Insertion Sort)

原理

逐一將未排序的元素取出，由後往前與已排序部分的元素比較，並插入到適當的位置（類似整理撲克牌）。

複雜度分析

- 時間複雜度：最佳 $O(n)$ / 平均 $O(n^2)$ / 最差 $O(n^2)$
- 空間複雜度： $O(1)$

模擬實作內容

實作上將選中的元素與左側已排序的元素依序比較，若左側較大則向右位移，直到找到插入點。

4. 合併排序法 (Merge Sort)

原理

採用分治法（Divide and Conquer），將陣列遞迴拆解至最小單位後，再依大小順序兩兩合併回原本的陣列。

複雜度分析

- 時間複雜度：最佳 $O(n \log n)$ / 平均 $O(n \log n)$ / 最差 $O(n \log n)$
- 空間複雜度： $O(n)$

模擬實作內容

透過遞迴函數處理拆解，並使用暫存陣列來存放合併過程中的排序結果，視覺化上呈現區塊式的合併。

5. [在此填入第五種] 排序法

原理

[請在此處補充該排序法的簡短原理說明]

複雜度分析

- 時間複雜度：[例如： $O(n \log n)$]
- 空間複雜度：[例如： $O(\log n)$]

模擬實作內容

[說明此演算法在網頁模擬中的呈現方式]

6. 綜合比較

排序演算法	平均時間複雜度	最佳時間複雜度	最差時間複雜度	空間複雜度	穩定性
Bubble Sort	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	穩定

排序演算法	平均時間複雜度	最佳時間複雜度	最差時間複雜度	空間複雜度	穩定性
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不穩定
Insertion Sort	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	穩定
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	穩定

7. 學習心得

透過本次模擬網頁的製作，我對於各種排序演算法在不同資料狀況下的執行效率有了更直觀的認識。除了理論層面的複雜度分析外，實作視覺化動畫更考驗了對非同步處理 (Async/Await) 的控制，以確保排序步驟能以肉眼可見的速度正確呈現。這次的經驗讓我深刻體會到資料結構與演算法在程式優化上的重要性。